

Estructura de Computadores

Práctica MIPS LABS

Curso 2023/2024

Documentación Interna

Autores:

Jorge Méndez Martínez y José Manuel Pecero Blanco

Grupo de prácticas:

Laboratorio 1

Versión de Vivado:

Xilinx Vivado 2015.1

Introducción.....	3
Sesión 1	4
1.1- Código del circuito “mux4a1_4bits”:	4
1.2- Código del circuito “Seldigit_CodLEDS”:	4
1.3- Análisis RTL del circuito “Seldigit_CodLEDS”:	6
1.4- Simulación del circuito “Seldigit_CodLEDS”:	7
1.5- Pregunta de la sesión:	7
Sesión 2	8
2.1- Código VHDL del circuito “ALU16bits”:	8
2.2- Captura de pantalla del resultado del análisis RTL del circuito “ALU16bits”	9
2.3- Código VHDL del testbench del circuito “ALU16bits”, modificado según las indicaciones del profesor	10
2.4- Resultados de la simulación del circuito “ALU16bits”	11
2.5- Código VHDL del testbench del circuito “registro_PC”	13
2.6- Resultados de la simulación del circuito “registro_PC”	15
Sesión 3	16
3.1- Código VHDL del testbench del circuito “Mem_la_256x8”	16
3.2- Resultados de la simulación del circuito “Mem_la_256x8”	17
3.3- Código VHDL del circuito “ram_datos”	18
3.4- Resultados de la simulación del circuito “ram_datos”	22
3.5- Pregunta de la sesión:	23
Sesión 4	24
4.1- Código VHDL del circuito “Mips_nucleo1” en el que se realiza la descripción estructural del procesador	24
4.2- Captura de pantalla del resultado del análisis RTL del circuito “Mips_nucleo1”	28
4.3- Resultados de la simulación del circuito “Mips_nucleo1”	30
4.4- Pregunta de la sesión:	32
Sesión 5	33
5.1- Captura de pantalla del resultado del análisis RTL del circuito “MIPS_Interface”	33
5.2- Resultados de la simulación del circuito “MIPS_Interface” (una captura de pantalla)	34
5.3- Captura del resultado de la síntesis e implementación (“Implemented Design”), mostrando la utilización de recursos en la FPGA.....	35
5.4- FPGA en las que aparezca visualizados:	36
5.5- Resumen final del proyecto	39
5.6- Pregunta de la sesión:	40

Introducción

En la asignatura de Estructura de Computadores, hemos hecho una práctica para desarrollar el entorno de MIPS LAB diseñando un procesador MIPS monociclo, usando el nuevo lenguaje de programación VHDL y otras herramientas como el programa VIVADO para modelar, simular e implementar en FPGA.

A lo largo de esta documentación interna, dividida en 5 sesiones de laboratorio, se detallará el proceso de desarrollo del procesador, desde la modelación en código VHDL, hasta la implementación hardware en la tarjeta Nexys4DDR.

Finalmente, hay que recalcar que este documento pretende recopilar y analizar de manera detallada cada etapa del proyecto, extrayendo también valiosas lecciones que puedan aportar a otros futuros proyectos.

Sesión 1

Durante esta primera sesión hemos estado viendo cómo diseñar distintos módulos ("mux4a1_4bits", "dec2a4" y "hex2led") codificándolos en VHDL a través de distintos procedimientos (creación de un archivo VHDL y edición de su contenido y a través de la importación de un archivo VHDL), continuando la sesión con la conexión de dichos módulos mediante VHDL, formando el circuito "Seldigit_CodLEDS" para posteriormente observar el análisis RTL del circuito y depurar posibles errores. Terminando finalmente la práctica con la simulación del circuito.

1.1- Código del circuito "mux4a1_4bits":

```
1. entity mux4a1_4bits is
2.     Port ( D0 : in STD_LOGIC_VECTOR (3 downto 0);
3.           D1 : in STD_LOGIC_VECTOR (3 downto 0);
4.           D2 : in STD_LOGIC_VECTOR (3 downto 0);
5.           D3 : in STD_LOGIC_VECTOR (3 downto 0);
6.           SEL : in STD_LOGIC_VECTOR (1 downto 0);
7.           S : out STD_LOGIC_VECTOR (3 downto 0));
8. end mux4a1_4bits;
9.
10.    architecture Behavioral of mux4a1_4bits is
11.
12.    begin
13.
14.        s <= D0 when SEL = "00" else
15.            D1 when SEL = "01" else
16.            D2 when SEL = "10" else
17.            D3 when SEL = "11" else
18.            X"0";
19.
20.    end Behavioral;
```

1.2- Código del circuito "Seldigit_CodLEDS":

```
21. entity Seldigit_CodLEDS is
22.     Port ( regs : in STD_LOGIC_VECTOR (15 downto 0);
23.           refresh : in STD_LOGIC_VECTOR (1 downto 0);
24.           anodos : out STD_LOGIC_VECTOR (3 downto 0);
25.           catodos : out STD_LOGIC_VECTOR (7 downto 0));
26. end Seldigit_CodLEDS ;
27.
28.    architecture Behavioral of Seldigit_CodLEDS is
29.    signal HEXled: STD_LOGIC_VECTOR(6 DOWNTO 0);
30.    signal punto: std_logic:='1';
31.    signal digHEX: STD_LOGIC_VECTOR(3 DOWNTO 0);
32.
33.    alias digit3: STD_LOGIC_VECTOR(3 DOWNTO 0) is REGS(15 DOWNTO
12);
```

```

34.  alias digit2: STD_LOGIC_VECTOR(3 DOWNTO 0) is REGS(11 DOWNTO
    8);
35.  alias digit1: STD_LOGIC_VECTOR(3 DOWNTO 0) is REGS(7 DOWNTO 4);
36.  alias digit0: STD_LOGIC_VECTOR(3 DOWNTO 0) is REGS(3 DOWNTO 0);
37.
38.  COMPONENT dec2a4
39.      PORT(
40.          d : IN std_logic_vector(1 downto 0);
41.          y : OUT std_logic_vector(0 to 3)
42.      );
43.  END COMPONENT;
44.
45.  COMPONENT mux4a1_4bits
46.      PORT(
47.          d0 : IN std_logic_vector(3 downto 0);
48.          d1 : IN std_logic_vector(3 downto 0);
49.          d2 : IN std_logic_vector(3 downto 0);
50.          d3 : IN std_logic_vector(3 downto 0);
51.          sel : IN std_logic_vector(1 downto 0);
52.          S : OUT std_logic_vector(3 downto 0)
53.      );
54.  END COMPONENT;
55.
56.  COMPONENT hex2led
57.      PORT(
58.          hex : IN std_logic_vector(3 downto 0);
59.          led : OUT std_logic_vector(6 downto 0)
60.      );
61.  END COMPONENT;
62.
63.  begin
64.      catodos <= HEXled & punto;
65.      Inst_dec2a4: dec2a4
66.      PORT MAP(
67.          d => refresh,
68.          y => anodos
69.      );
70.
71.      Inst_mux4a1_4bits: mux4a1_4bits
72.      PORT MAP(
73.          d0 => digit3,
74.          d1 => digit2,
75.          d2 => digit1,
76.          d3 => digit0,
77.          sel => refresh,
78.          s => digHEX
79.      );
80.
81.      Inst_hex2led: hex2led
82.      PORT MAP(
83.          hex => digHEX,

```

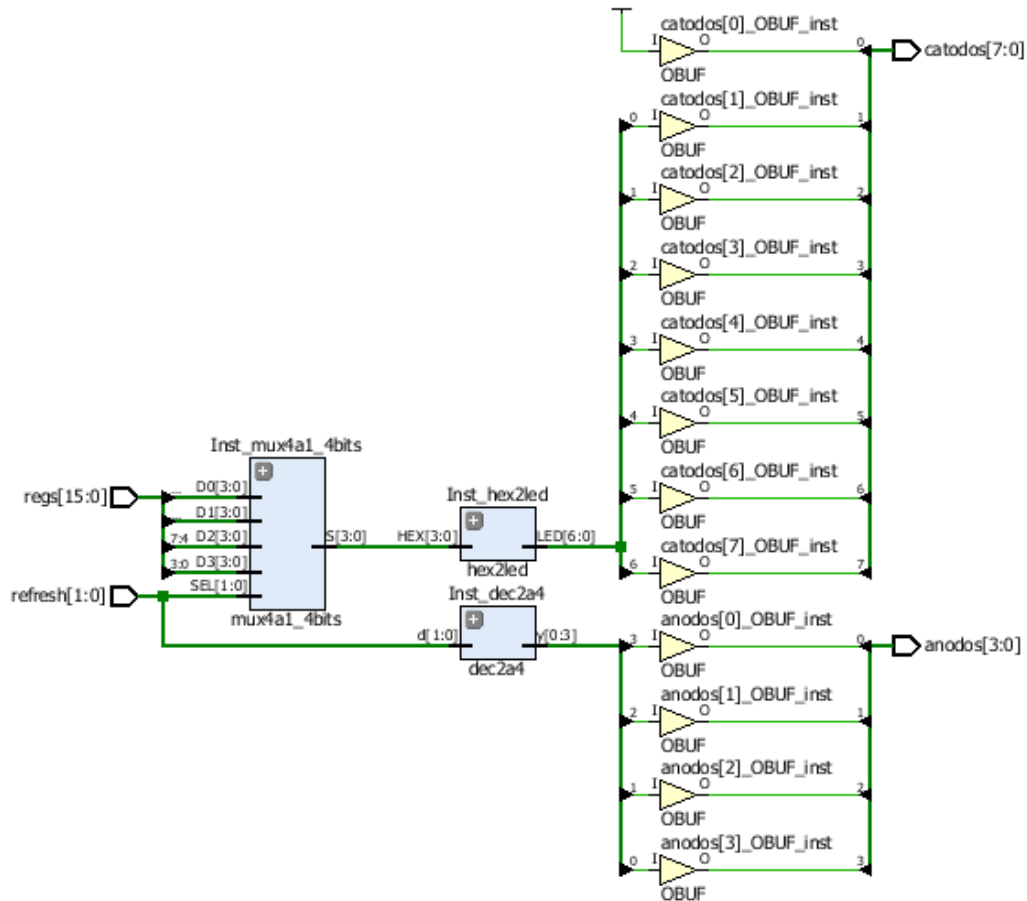
```

84.         led => HEXled
85.     );
86.
87.     end Behavioral;

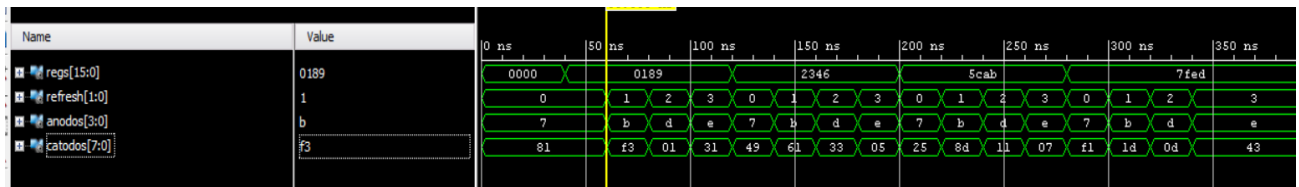
```

1.3- Análisis RTL del circuito “Seldigit_CodLEDS”:

Tras realizar el análisis RTL del circuito “Seldigit_CodLEDS”, se muestra el siguiente esquema lógico.



1.4- Simulación del circuito “Seldigit_CodLEDS”:

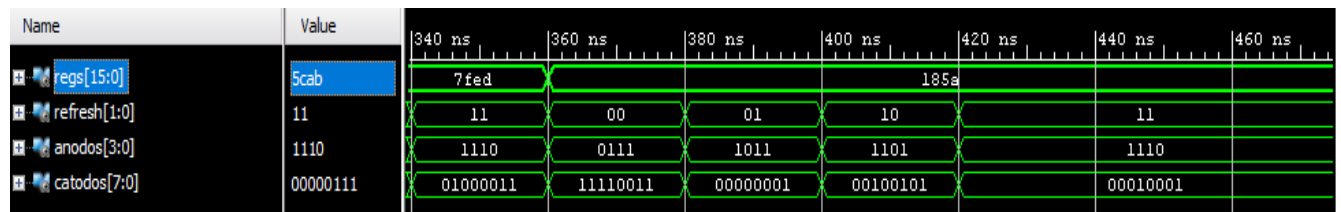


1.5- Pregunta de la sesión:

¿Qué valores binarios en cátodos se tienen q generar para 185A? Y para dicha pregunta obtuvimos la siguiente captura añadiendo al circuito “TB_Seldigit_codleds” el siguiente código:

```
88.     regs <= X"185A";
89.     for i in 0 to 3 loop
90.         refresh <= conv_std_logic_vector(i,2);
91.         wait for 20 ns;
```

Obteniendo así la captura de la simulación, en donde podemos ver los valores en binario necesarios para generar 185A (mostrado en regs en hexadecimal).



Sesión 2

En esta segunda sesión hemos implementado y simulado dos componentes del procesador, estos son "ALU16bits.vhd" y "registro_PC.vhd", codificando el circuito del primer componente nombrado, obteniendo el análisis RTL de dicho circuito y simulándolo después de hacerle las modificaciones indicadas por el profesor.

2.1- Código VHDL del circuito "ALU16bits":

Para el circuito nombrado hemos usado el siguiente código:

```
92.     library IEEE;
93.     use IEEE.STD_LOGIC_1164.ALL;
94.     use IEEE.STD_LOGIC_ARITH.ALL;
95.     use IEEE.STD_LOGIC_SIGNED.ALL;
96.
97.     entity ALU16bits is
98.         Port ( A : in STD_LOGIC_VECTOR (15 downto 0);
99.               B : in STD_LOGIC_VECTOR (15 downto 0);
100.            con : in STD_LOGIC_VECTOR (2 downto 0);
101.            Z : out STD_LOGIC;
102.            res : out STD_LOGIC_VECTOR (15 downto 0));
103.     end ALU16bits;
104.
105.     architecture Behavioral of ALU16bits is
106.         signal R: std logic vector (15 downto 0);
107.     begin
108.
109.         process (A, B, con)
110.         begin
111.             case con is
112.                 when "000"=>
113.                     R <= (A and B);
114.                 when "001" =>
115.                     R <= (A or B);
116.                 when "010" =>
117.                     R <= (A + B);
118.                 when "011" =>
119.                     R <= (A xor B);
120.                 when "100" =>
121.                     R <= (A nor B);
122.                 when "110" =>
123.                     R <= (A - B);
124.                 when "111" =>
125.                     if (A < B) then
```

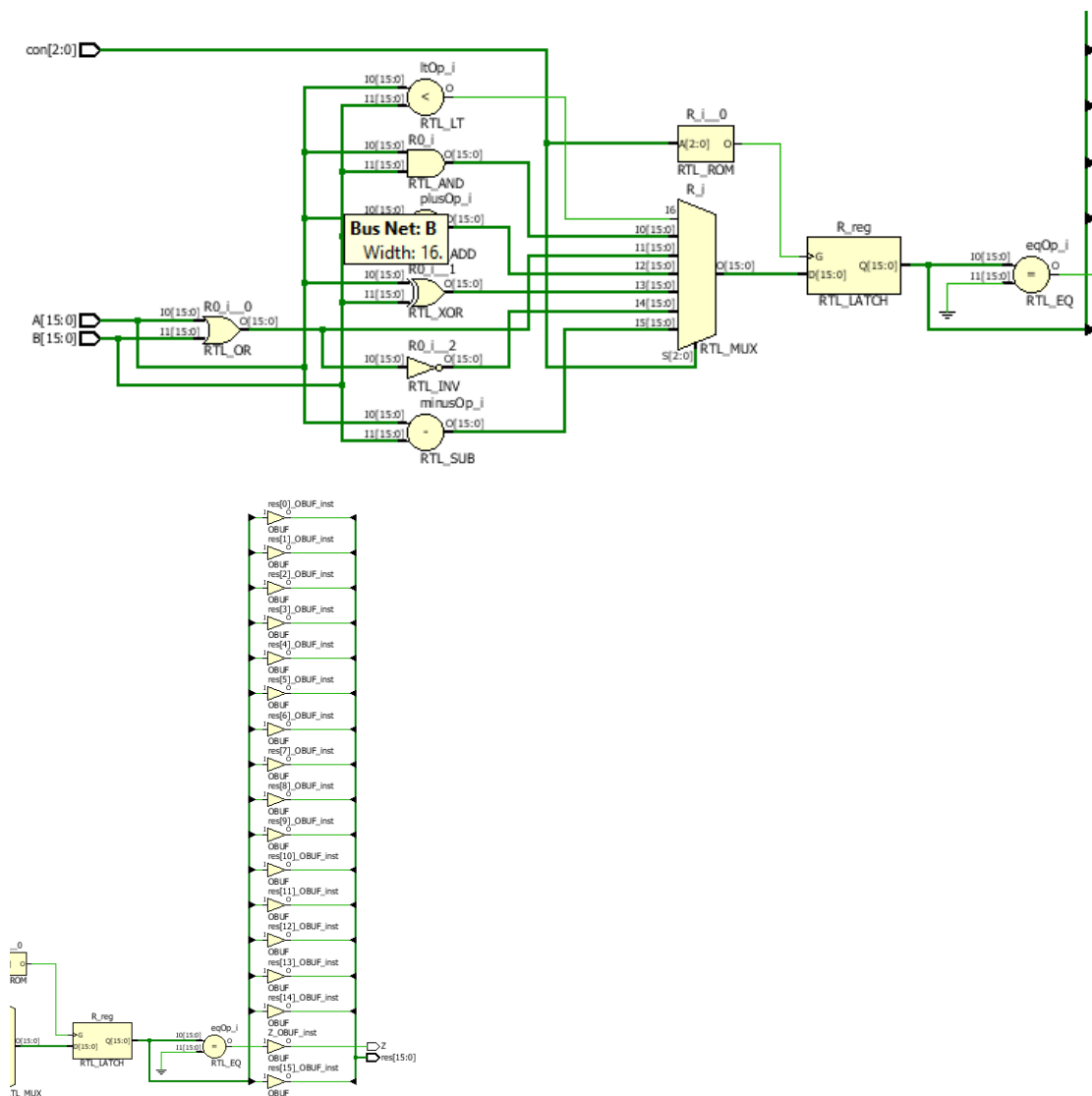
```

126.             R <= X"0001";
127.         else
128.             R <= X"0000";
129.         end if;
130.     when others => null;
131. end case;
132. end process;
133. res <= R;
134. Z <= '1' when R = X"0000" else '0';
135. end Behavioral;

```

2.2- Captura de pantalla del resultado del análisis RTL del circuito "ALU16bits"

Estamos trabajando en Vivado 2015.1, por ello necesitamos hacer dos capturas del análisis RTL



2.3- Código VHDL del testbench del circuito "ALU16bits", modificado según las indicaciones del profesor

Para el código del testbench del circuito indicado, tuvimos que hacerle modificaciones para realizarse en primer lugar todas las operaciones aritméticas y después las lógicas soportadas por la ALU, obteniendo lo siguiente:

```
136.    library IEEE;
137.    use IEEE.STD_LOGIC_1164.ALL;
138.
139.    entity TB_ALU16bits is
140.    end TB_ALU16bits;
141.
142.    architecture Behavioral of TB_ALU16bits is
143.
144.    COMPONENT ALU16bits
145.        port(
146.            A : in STD_LOGIC_VECTOR(15 downto 0);
147.            B : in STD_LOGIC_VECTOR(15 downto 0);
148.            con : in STD_LOGIC_VECTOR(2 downto 0);
149.            z : out STD_LOGIC;
150.            res : out STD_LOGIC_VECTOR(15 downto 0)
151.        );
152.    END COMPONENT;
153.
154.    --input
155.    signal A : STD_LOGIC_VECTOR (15 downto 0) := X"000C";
156.    signal B : STD_LOGIC_VECTOR (15 downto 0) := X"000F";
157.    signal con : STD_LOGIC_VECTOR (2 downto 0) := (others => '1');
158.    --output
159.    signal z : STD_LOGIC;
160.    signal res : STD_LOGIC_VECTOR (15 downto 0);
161.
162.    begin
163.        uut: ALU16bits PORT MAP(
164.            A => A,
165.            B => B,
166.            con => con,
167.            z => z,
168.            res => res
169.        );
170.
171.        --stimulus process
172.        stim_proc: process
173.        begin
174.            -- primer set de pruebas, a ordenar de acuerdo a las
            -- especificaciones de la práctica
175.            wait for 20 ns;
176.            con <= "010"; -- SUMA
177.            wait for 10 ns;
178.            con <= "110"; -- RESTA
179.            wait for 10 ns;
```

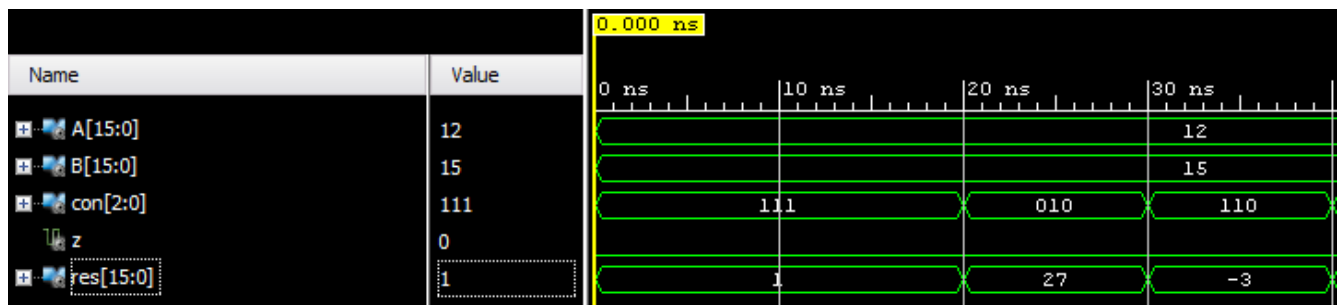
```

180.         con <= "000"; -- AND
181.         wait for 10 ns;
182.         con <= "001"; -- OR
183.         wait for 10 ns;
184.         con <= "011"; -- XOR
185.         wait for 10 ns;
186.         con <= "100"; -- NOR
187.         wait for 10 ns;
188.         -- segundo set de pruebas, probamos con un valor menor
           del segundo operando
189.         B <= X"0004";
190.         con <= "010";
191.         wait for 10 ns;
192.         con <= "110";
193.         wait for 10 ns;
194.         con <= "111";
195.         wait;
196.     end process;
197.
198. end Behavioral;

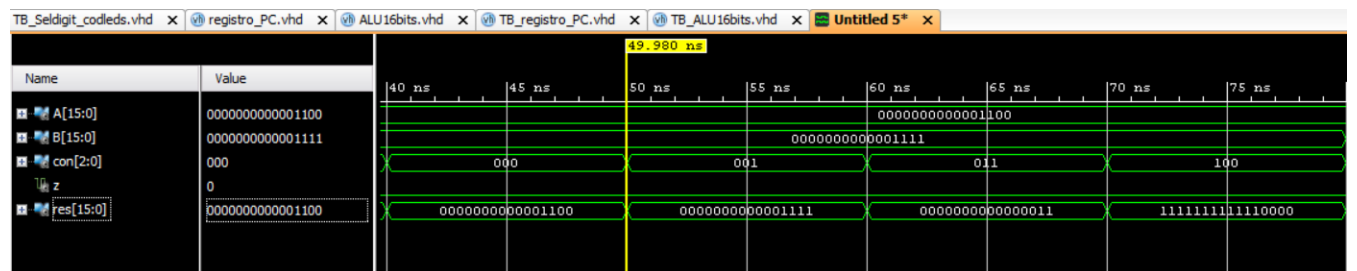
```

2.4- Resultados de la simulación del circuito “ALU16bits”

Como observamos la simulación realiza bien las operaciones, lo podemos comprobar viendo los valores hexadecimales en esta primera captura, A tiene valor 12 y B 15, al hacer la suma obtendríamos 27 y al hacer la resta -3, valores q aparecen en la simulación.



Para el resto de las operaciones lógicas podemos comprobar viendo los números binarios que se realizan correctamente.

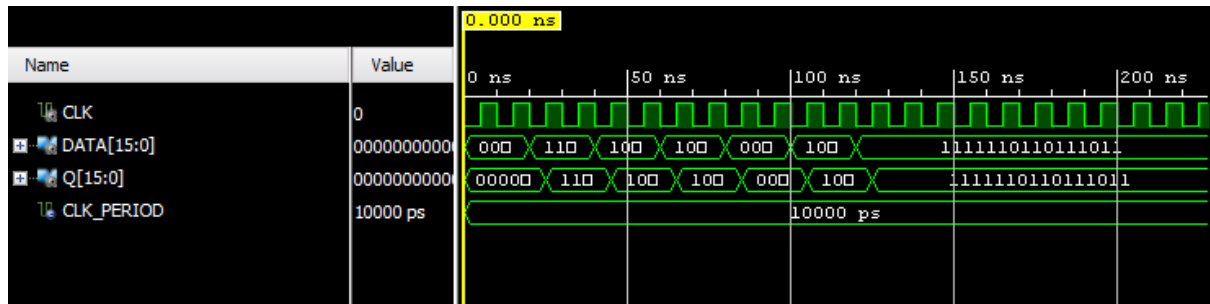


2.5- Código VHDL del testbench del circuito “registro_PC”

```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity TB_registro_PC is
5. end TB_registro_PC;
6.
7. architecture Behavioral of TB_registro_PC is
8.
9. COMPONENT registro_PC
10.     port (
11.         CLK : in std_logic;
12.         DATA : in std_logic_vector(15 downto 0);
13.         Q : out std_logic_vector(15 downto 0)
14.     );
15. END COMPONENT;
16.
17. --inputs
18. signal CLK : std_logic := '0';
19. signal DATA : std_logic_vector(15 downto 0) := (others => '0');
20.
21. --output
22. signal Q: std_logic_vector(15 downto 0);
23.
24. --constantes
25. constant CLK_PERIOD: time:= 10 ns;
26.
27. begin
28.
29.     uut: registro_PC PORT MAP(
30.         CLK => CLK,
31.         DATA => DATA,
32.         Q => Q
33.     );
34.
35.     -- Clock process definitions
36.     CLK_process: process
37.     begin
38.         CLK <= '0';
39.         wait for CLK_period/2;
40.         CLK <= '1';
41.         wait for CLK_period/2;
42.     end process;
43.
44.     -- Stimulus process definitions
45.     stim_proc: process
```

```
46.         begin
47.             wait for CLK_period*2;
48.             DATA <= (others => '1');
49.             wait for CLK_period*2;
50.             DATA <= X"BBCF";
51.             wait for CLK_period*2;
52.             DATA <= X"AACC";
53.             wait for CLK_period*2;
54.             DATA <= X"2468";
55.
56.             wait for CLK_period*2;
57.             DATA <= X"AC13";
58.             wait for CLK_period*2;
59.             DATA <= X"FDBB";
60.
61.             wait;
62.         end process;
63. end Behavioral;
```

2.6- Resultados de la simulación del circuito “registro_PC”



Sesión 3

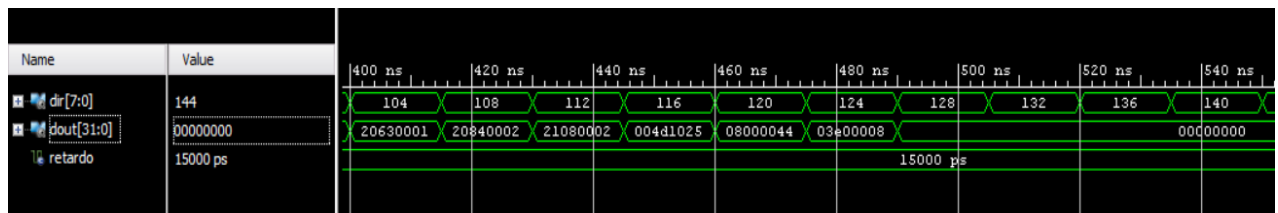
3.1- Código VHDL del testbench del circuito “Mem Ia 256x8”

```
199. library IEEE;
200. use IEEE.STD_LOGIC_1164.ALL;
201. use IEEE.STD_LOGIC_ARITH.ALL;
202.
203. entity TB_Mem_Ia_256x8 is
204. -- Port ( );
205. end TB_Mem_Ia_256x8;
206.
207. architecture Behavioral of TB_Mem_Ia_256x8 is
208.
209. COMPONENT Mem_Ia_256x8
210.     PORT (
211.         dir: in std_logic_vector (7 downto 0);
212.         dout: out std_logic_vector (31 downto 0));
213. end COMPONENT;
214.
215.     signal dir: std_logic_vector(7 downto 0) := (others=>'0');
216.     signal dout: std_logic_vector(31 downto 0);
217.     constant retardo: time:= 15 ns;
218.
219. begin
220.     uut: Mem_Ia_256x8 PORT MAP(
221.         dir => dir,
222.         dout => dout
223.     );
224.
225.     -- Stimulus process
226.     stim_proc: process
227.         variable dir_palabra: std_logic_vector(5 downto 0); --
228.         63 instrucciones
229.         begin
230.             -- hold reset state for 10 ns.
231.             wait for 10 ns;
232.             for i in 0 to 63 loop
233.                 dir_palabra:= conv_std_logic_vector(i,6);
234.                 dir <= dir_palabra & "00";
235.                 wait for retardo;
236.             end loop;
237.             wait;
238.         end process;
239. end Behavioral;
```

3.2- Resultados de la simulación del circuito “Mem_la_256x8”

Dir-> decimal sin signo

Dout -> hexadecimal o binario



¿En qué nano segundos se leen estas instrucciones? (en qué momentos se leen esas instrucciones, captura)

1.- Jal rutina

Esta instrucción se encuentra en la dirección de memoria 32 y se leen entre los 130 y 145 ns

2.- lw \$5, 0(\$4)

Esta instrucción se realiza en la dirección de memoria 76 y se leen entre 295 y 310 ns

3.- sw 13, 0(\$8)

Esta instrucción se realiza en la dirección de memoria 100 y se leen entre 385 y 400 ns

4.- Or \$2, \$2, \$13

Esta instrucción se realiza en la dirección de memoria 116 y se leen entre 445 y 460 ns

Para realizar este apartado nos hemos ido fijando dentro código del circuito “Mem_la_256x8” en las direcciones de memoria en las cuales se realizaban las instrucciones para después observar la simulación del testbench y ver los nano segundos en los que se leían

3.3- Código VHDL del circuito "ram_datos"

La especificación del código VHDL de la memoria del alumno Jorge Méndez viene dado por los siguientes datos:

Direcciones 12 – 13, A(0) = +14 (000E)

Direcciones 14 – 15, A(1) = + JO = 19 + 31 = 50 (0032)

Direcciones 16 – 17, A(2) = - (RG) = -(37 + 13) = -50 (FFCE)

Direcciones 18 – 19, A(3) = - (EM) = -(9 + 25) = -34 (FFDE)

Direcciones 20 – 21, A(4) = + EN = 9 + 27 = 36 (0024)

Direcciones 22 – 23, A(5) = -100*K = -100 (FF9C)

Código del alumno Jorge Méndez:

```
240.    library IEEE;
241.    use IEEE.std_logic_1164.all;
242.    use IEEE.STD_LOGIC_ARITH.ALL;
243.    use IEEE.STD_LOGIC_UNSIGNED.ALL;
244.
245.    entity ram_datos is
246.        port (
247.            WE : in std_logic;
248.            CLK : in std_logic;
249.            ADDR : in std_logic_vector(7 downto 0);
250.            DATAin : in std_logic_vector(15 downto 0);
251.            DATAOUT : out std_logic_vector(15 downto 0)
252.        );
253.    end entity;
254.
255.
256.    architecture ram_arch of ram_datos is
257.
258.        -- type ram_mem_type is array (63 downto 0) of
259.        std_logic_vector(7 downto 0);
260.        -- se ha cambiado a sentido ascendente del tipo
261.        type ram_mem_type is array (0 to 255) of
262.        std_logic_vector(7 downto 0);
263.
264.        -- número elementos del array : 6
265.        -- elementos del array: +1, +13, +7, -40, -18, -Ngrup*100
266.
267.        signal ram_mem : ram_mem_type := (
268.            6 => X"00",
269.            7 => X"0C", -- dir de inicio del array1
270.            8 => X"00",
271.            9 => X"20", -- dir. de inicio del array2
```

```

270.             10 => X"00",
271.             11 => X"06",
272.             -- num. de elementos en 10 (ByteH) y 11(ByteL)
273.             -- DATOS ASOCIADOS AL NOMBRE a partir de la dirección
274.             14
275.             -- NOMBRE correspondiente a estos datos:
276.             12 => X"00",
277.             13 => X"0E", -- +14
278.             14 => X"00",
279.             15 => X"32", -- +50
280.             16 => X"FF",
281.             17 => X"CE", -- -50
282.             18 => X"FF",
283.             19 => X"DE", -- -34
284.             20 => X"00",
285.             21 => X"24", -- +36
286.             22 => X"FF",
287.             23 => X"9C", -- -100
288.             others => X"00");
289.
290.
291.     begin
292.
293.         process (CLK)
294.         begin
295.             if rising_edge(CLK) then
296.                 if (WE = '1') then
297.                     ram_mem(CONV_INTEGER(ADDR)) <=
298.                         DATAin(15 downto 8);
299.                     ram_mem(CONV_INTEGER(ADDR + 1))
300.                         <= DATAin(7 downto 0);
301.                 end if;
302.             end if;
303.         end process;
304.
305.         bytealto <= ram_mem(CONV_INTEGER(ADDR));
306.         bytebajo <= ram_mem(CONV_INTEGER(ADDR + 1));
307.         Dataout <= bytealto & bytebajo;
308.     end architecture;

```

La especificación del código VHDL de la memoria del alumno José Manuel Pecero viene dado por los siguientes datos:

Direcciones 12 – 13, A(0) = +14 (000E)

Direcciones 12 – 13, A(1) = +10 = +(19+31) = +50 (0032)

Direcciones 12 – 13, A(2) = -SE = -(39+9) = -48 (FFD0)

Direcciones 12 – 13, A(3) = -MA = -(25+1) = -26 (FFE6)

Direcciones 12 – 13, A(4) = +NU = +(27+43) = +70 (0046)

Direcciones 12 – 13, A(5) = -100*K = -100 (FF9C)

Código del alumno José Manuel Pecero:

```
1. library IEEE;
2. use IEEE.std_logic_1164.all;
3. use IEEE.STD_LOGIC_ARITH.ALL;
4. use IEEE.STD_LOGIC_UNSIGNED.ALL;
5.
6. entity ram_datos is
7.     port (
8.         WE : in std_logic;
9.         CLK : in std_logic;
10.        ADDR : in std_logic_vector(7 downto 0);
11.        DATAin : in std_logic_vector(15 downto 0);
12.        DATAOUT : out std_logic_vector(15 downto 0)
13.    );
14. end entity;
15.
16.
17. architecture ram_arch of ram_datos is
18.
19.     -- type ram_mem_type is array (63 downto 0) of
20.     std_logic_vector(7 downto 0);
21.     -- se ha cambiado a sentido ascendente del tipo
22.     type ram_mem_type is array (0 to 255) of
23.     std_logic_vector(7 downto 0);
24.
25.     -- número elementos del array : 6
26.     -- elementos del array: +1, +13, +7, -40, -18, -Ngrup*100
27.
28.     signal ram_mem : ram_mem_type := (
29.         6 => X"00",
30.         7 => X"0C", -- dir de inicio del array1
31.         8 => X"00",
32.         9 => X"20", -- dir. de inicio del array2
33.         10 => X"00",
34.         11 => X"06",
35.         -- num. de elementos en 10 (ByteH) y 11(ByteL)
36.         -- DATOS ASOCIADOS AL NOMBRE a partir de la dirección
37.         14
38.         -- NOMBRE correspondiente a estos datos: ALBERTO B
39.         12 => X"00",
```

```

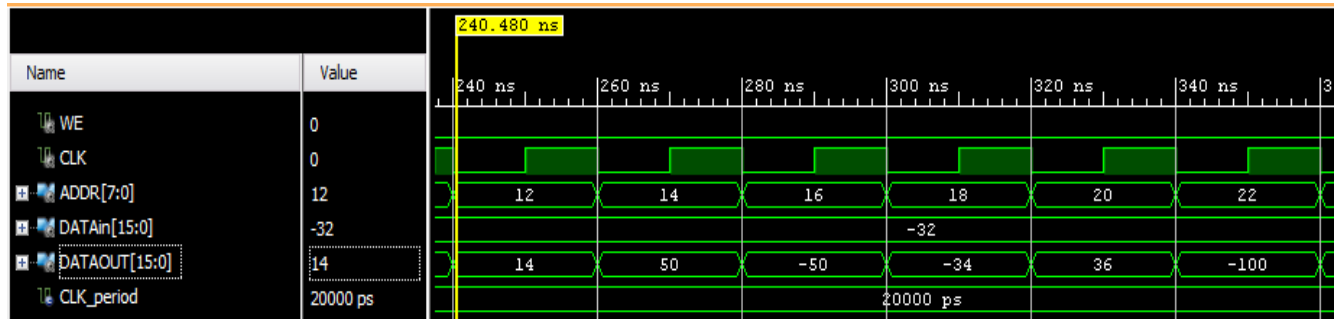
37.             13 => X"0E", -- +14
38.             14 => X"00",
39.             15 => X"32", -- +50
40.             16 => X"FF",
41.             17 => X"D0", -- -48
42.             18 => X"FF",
43.             19 => X"E6", -- -26
44.             20 => X"00",
45.             21 => X"46", -- +70
46.             22 => X"FF",
47.             23 => X"9C", -- - (1*100) = -
100 (para grupo 1)
48.             others => X"00");
49.
50.             signal bytealto, bytebajo: std logic vector(7 downto
0);
51.
52.             begin
53.
54.                 process (CLK)
55.                     begin
56.                         if rising_edge(CLK) then
57.                             if (WE = '1') then
58.                                 ram_mem(CONV_INTEGER(ADDR)) <=
DATAin(15 downto 8);
59.                                 ram_mem(CONV_INTEGER(ADDR + 1))
<= DATAin(7 downto 0);
60.                             end if;
61.                         end if;
62.                     end process;
63.
64.                     bytealto <= ram_mem(CONV_INTEGER(ADDR));
65.                     bytebajo <= ram_mem(CONV_INTEGER(ADDR + 1));
66.                     Dataout <= bytealto & bytebajo;
67.
68.             end architecture;

```

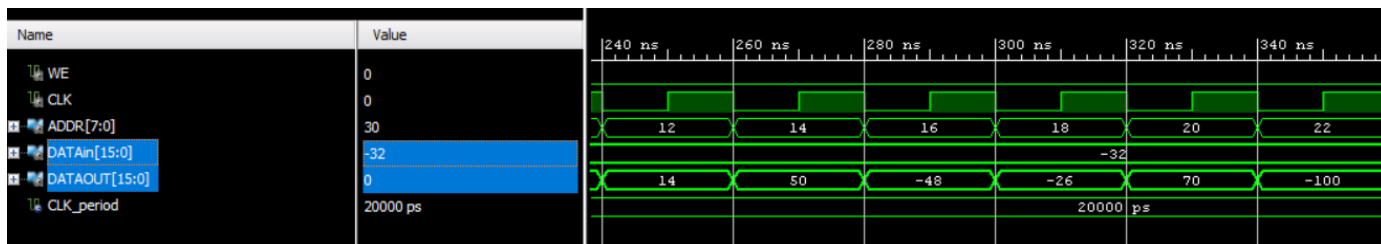
3.4- Resultados de la simulación del circuito “ram_datos”

Simulamos el circuito “Mem_la_256x8” con los 8 primeros caracteres de nuestro nombre codificados en el circuito “ram_datos”, de tal forma que obtendremos:

Captura con los caracteres “Jorge Men”.

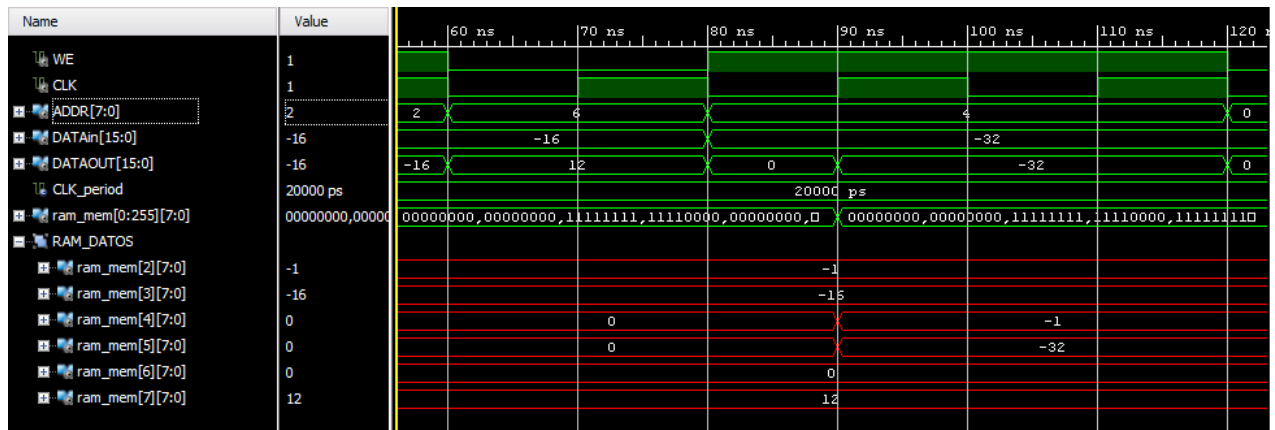


Captura con los caracteres “Jose Manu”.



3.5- Pregunta de la sesión:

Como podemos observar en las direcciones 2 y 3, no se escribe -8 porque se está escribiendo sobre las mismas direcciones un -16, reemplazando así a ese -8 por un -16. Después deshabilitamos la habilitación de escritura, por eso en las direcciones 6 y 7 no se escribe ningún 32 y se pone por defecto 0.



Sesión 4

En esta cuarta sesión a partir del modelo incompleto q se nos proporciona, completaremos el código del procesador "Mips_Nucleo1" utilizando todos sus componentes. Posteriormente comprobando que el código está bien implementado seleccionándolo como "Set as Top" y realizando un análisis RTL.

Finalmente verificaremos que realmente hemos hecho todas las conexiones bien simulando el archivo, terminando la sesión con una pregunta opcional proporcionada por el profesor.

4.1- Código VHDL del circuito "Mips_nucleo1" en el que se realiza la descripción estructural del procesador

Para este apartado nos hemos limitado a escribir el código del circuito indicado según los comentarios nos indicaban.

```
1. -- RELLENAR-----
2. -- declarar el resto de señales que sean necesarias
3. -- 1) definir la señal correspondiente al contenido de rs leído del banco de registros, de
   nombre "RSDData" (16 bits)
4. -- 2) definir la señal correspondiente al contenido de rt leído del banco de registros, de
   nombre "RTData" (16 bits)
5. -- 3) definir la señal para la segunda entrada de la ALU, de nombre "RTData_inm" (16 bits)
6. -- 4) definir la señal de salida de la ALU, de nombre "ALURes" (16 bits)
7. -- 5) definir la señal salida del control de la ALU que define la operación aritmetico-logica a
   realizar, de nombre "ALUControl" (3 bits)
8. -- -----
9. signal RSDData: STD_LOGIC_VECTOR (15 DOWNT0 0);
10.    signal RTData: STD_LOGIC_VECTOR (15 DOWNT0 0);
11.    signal RTData_inm: STD_LOGIC_VECTOR (15 DOWNT0 0);
12.    signal ALURes: STD_LOGIC_VECTOR (15 DOWNT0 0);
13.    signal ALUControl: STD_LOGIC_VECTOR (2 DOWNT0 0);
```

```
14. -- RELLENAR-----
15. -- Declarar un ALIAS para la señal de control MemWrite (1 bit), denominada "MemWrite". Es
   necesario asociar el alias a parte del vector de señales de control (vector_control)
16. -- MemWrite corresponde al bit 4 de vector_control, es decir, vector_control(4)
17. -- alias MemWrite:.....etc
18. -- -----
19.    alias MemWrite: std_logic is vector_control (4);
20.
21.    alias Branch: std_logic is vector_control(3);
22.    alias sel: std_logic is vector_control(2);
```

```
23.      alias PCSrc: std_logic_vector(1 downto 0) is vector_control(1
      downto 0);
```

```
24. -- DECLARACIÓN DE COMPONENTES
```

```
25. -- RELLENAR-----
```

```
26. -- 1) Declaración del componente ALU16bits (ver la declaración de la entidad en
      ALU16bits.vhd)
```

```
27. -- 2) Declaración del componente ALUcontrol_0 (ver la declaración de la entidad en
      ALUcontrol_0.vhd)
```

```
28. -- 3) Declaración del componente ram_datos (ver la declaración de la entidad en
      ram_datos.vhd)
```

```
29. -----
```

```
30.      COMPONENT ALU16bits
```

```
31.      port (
```

```
32.      A : in std_logic_vector (15 downto 0);
```

```
33.      B : in std_logic_vector (15 downto 0);
```

```
34.      con : in std_logic_vector (2 downto 0);
```

```
35.      Z : out std_logic;
```

```
36.      res : out std_logic_vector (15 downto 0)
```

```
37.      );
```

```
38.      end COMPONENT;
```

```
39.
```

```
40.      COMPONENT ALUcontrol_0
```

```
41.      port (
```

```
42.      ALUop: in std_logic_vector (1 downto 0);
```

```
43.      Func : in std_logic_vector (5 downto 0);
```

```
44.      cont_ALU : out std_logic_vector (2 downto 0)
```

```
45.
```

```
46.      );
```

```
47.      end COMPONENT;
```

```
48.
```

```
49.      COMPONENT ram_datos
```

```
50.      port (
```

```
51.      WE : in std_logic;
```

```
52.      CLK : in std_logic;
```

```
53.      ADDR : in std_logic_vector (7 downto 0);
```

```
54.      DATAin : in std_logic_vector (15 downto 0);
```

```
55.      DATAOUT : out std_logic_vector 15 downto 0)
```

```
56.      );
```

```
57.      end COMPONENT;
```

```

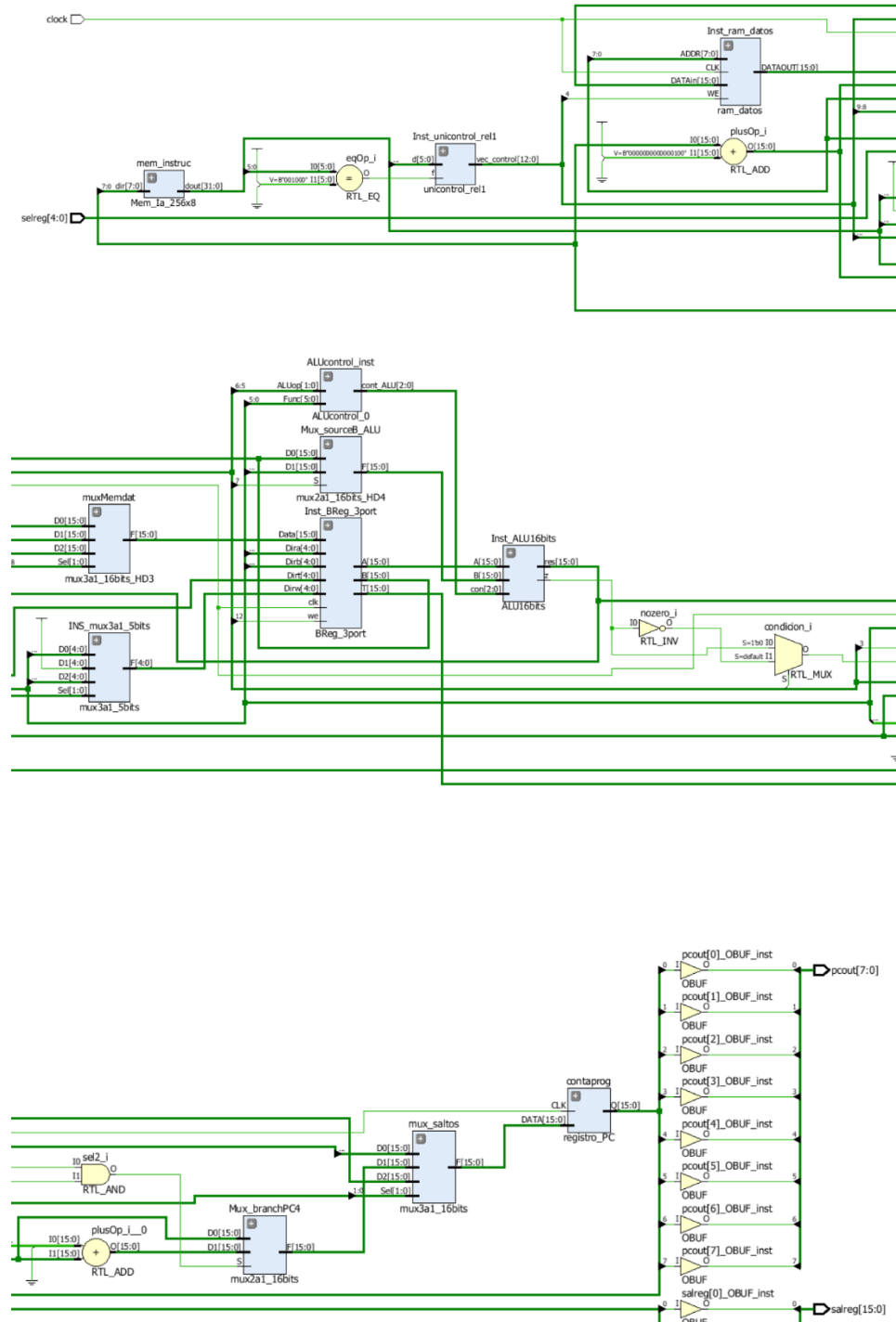
58. -- -----
59. -- INSTANCIAR Los componentes restantes del procesador a continuación
60. -- -----
61. -- RELLENAR-----
62. -- 1) Instanciar el componente mux2A1_16bits utilizado para decidir el segundo operando de
    la ALU (nombre de instancia "Mux_sourceB_ALU")
63. -- El mapeo debe realizarse D0 => RTData, D1 => dir_inm, S => ALUsrc, F => RTData_inm
64. -- 2) Instanciar el componente ALU16bits (nombre de instancia "Inst_ALU16bits")
65. -- El mapeo puerto-señal debe realizarse: A => RSData, B => RTData_inm, con => ALUControl,
    z => zero, res => ALURes
66. -- 3) Instanciar el componente ALUcontrol_0 (nombre de instancia "ALUcontrol_inst")
67. -- El mapeo debe realizarse ALUOp => ALU_Op, Func => funcion, cont_ALU => ALUControl
68. -- 4) Instanciar el componente ram_datos (nombre de instancia "Inst_ram_datos")
69. -- El mapeo debe realizarse WE => MemWrite, CLK => clock, ADDR => dirMdat, DATAin =>
    RTData, DATAOUT => salmemdat
70. -- -----
71.
72.     Mux_sourceB_ALU: mux2A1_16bits
73.     PORT MAP (
74.         D0 => RTData,
75.         D1 => dir_inm,
76.         S => ALUsrc,
77.         F => RTData_inm
78.     );
79.
80.     Inst_ALU16bits: ALU16bits
81.     PORT MAP(
82.         A => RSData,
83.         B => RTData_inm,
84.         con => ALUControl,
85.         z => zero,
86.         res => ALURes
87.     );
88.
89.     ALUcontrol_inst: ALUcontrol_0
90.     PORT MAP (
91.         ALUOp => ALU_Op,
92.         Func => funcion,
93.         cont_ALU => ALUControl
94.     );
95.
96.     Inst_ram_datos: ram_datos
97.     PORT MAP (
98.         WE => MemWrite,
99.         CLK => clock,

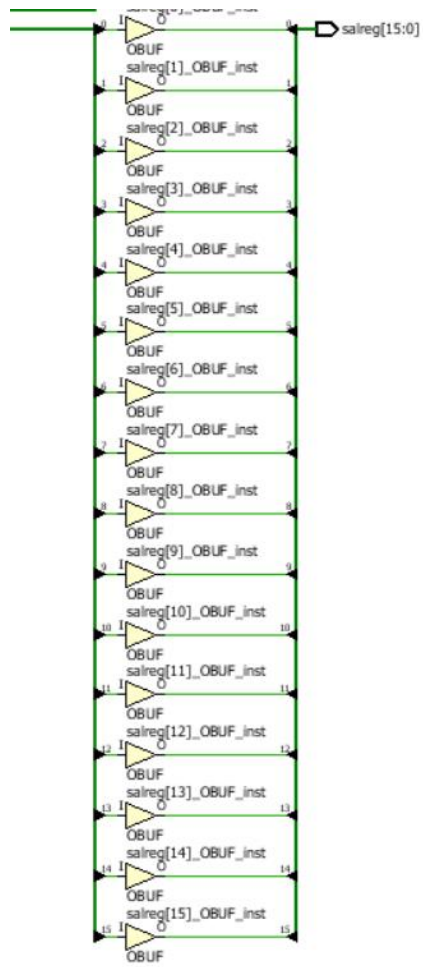
```

```
100.          ADDR => dirMdat,  
101.          DATAin => RTData,  
102.          DATAOUT => salmemdat  
103.      );
```

4.2- Captura de pantalla del resultado del análisis RTL del circuito “Mips_nucleo1”

Tras introducir el código del circuito “Mips_nucleo1” anteriormente indicado obtenemos el siguiente análisis RTL:





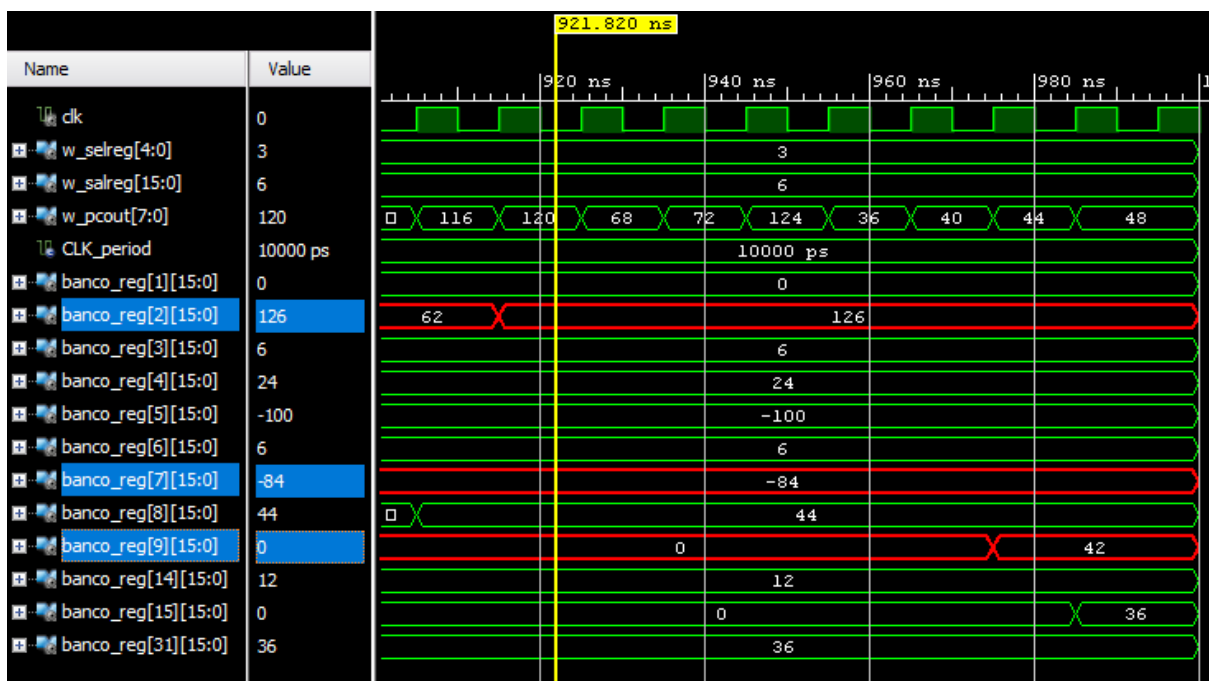
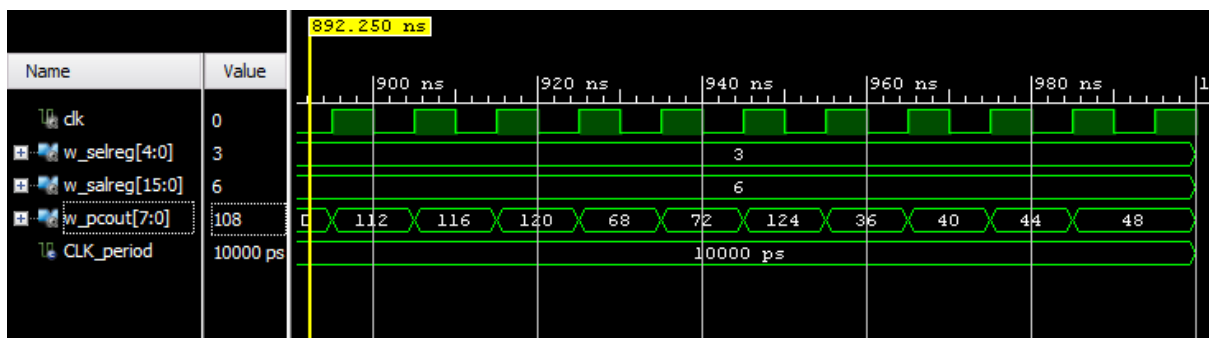
4.3- Resultados de la simulación del circuito “Mips nucleo1”

En esta simulación nos fijaremos principalmente en los registros \$2, \$7 y \$9, \$2 en primer lugar hará operaciones OR con todos los valores almacenados en el array1 (encontrado en el registro \$4), \$7 contendrá la suma de todos los valores anteriormente mencionados y por último el registro \$9 sumará el valor final del registro \$2 y \$7.

Por todo esto cada alumno tendrá que aportar una captura de su simulación ya que es única dependiendo de los valores del array1 (contiene los valores de los 8 primeros caracteres del nombre alumno).

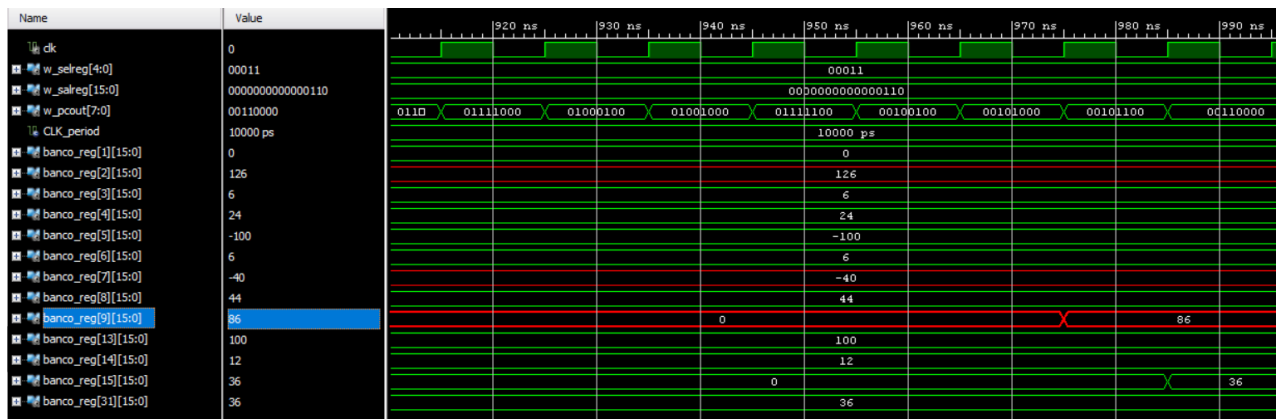
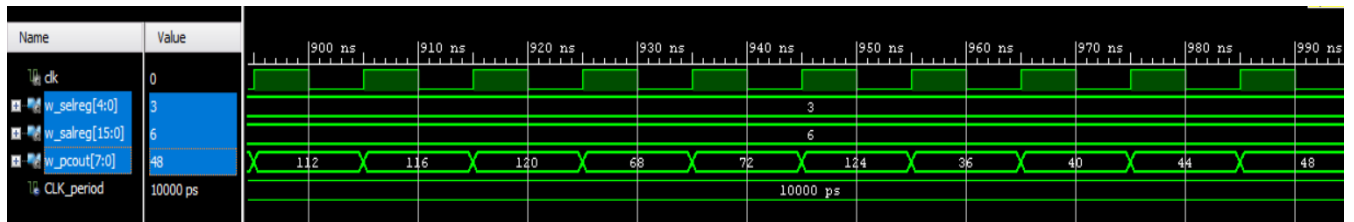
Simulación alumno Jorge Méndez:

En este caso podemos ver que \$2= 126, \$7= -84 y \$9= 42, por tanto, se realizan correctamente las operaciones.



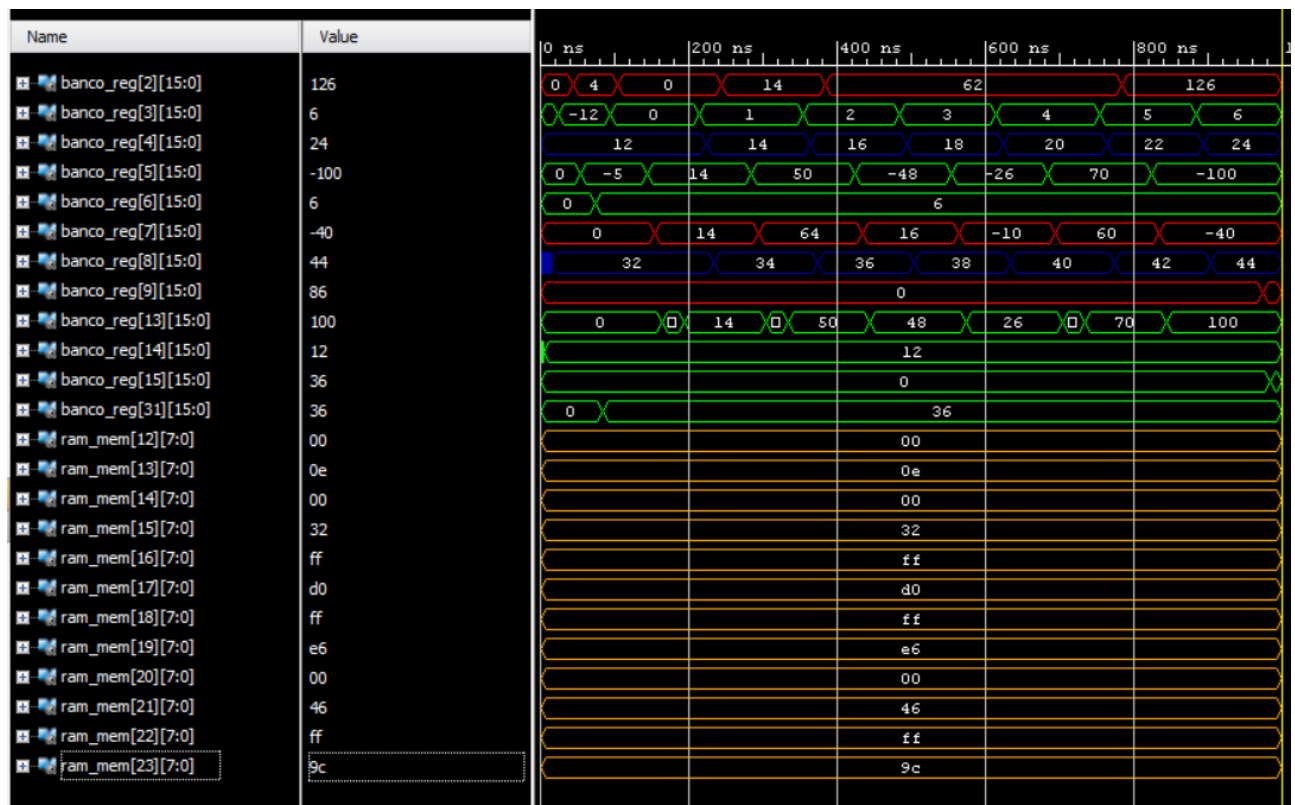
Simulación alumno José Manuel Pecero:

En este caso podemos ver que \$2= 126, \$7= -40 y \$9= 86, por tanto, se realizan correctamente las operaciones.



4.4- Pregunta de la sesión:

Contenido del array1.

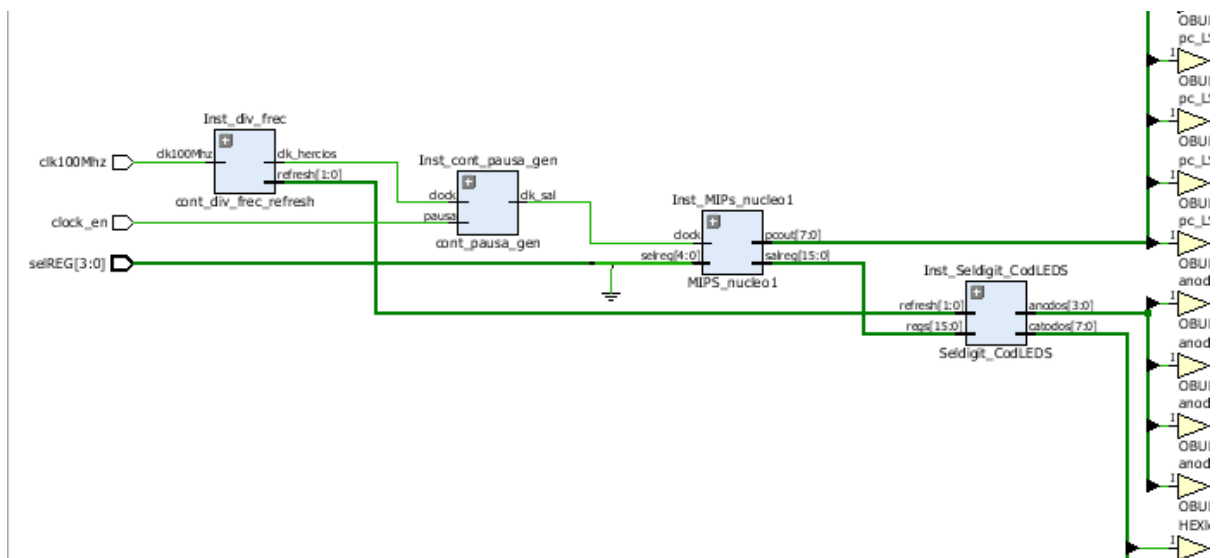


Sesión 5

En esta sesión después de haber implementado el circuito del procesador MIPS lo implementaremos en hardware configurando el chip de la FPGA de la tarjeta Nexys4DDR, utilizando como interfaz el módulo "Seldigit_CodLEDS.vhd" complementado con los módulos adicionales "cont_div_frec_refresh.vhd" y "cont_pausa_gen.vhd".

5.1- Captura de pantalla del resultado del análisis RTL del circuito "MIPS Interface"

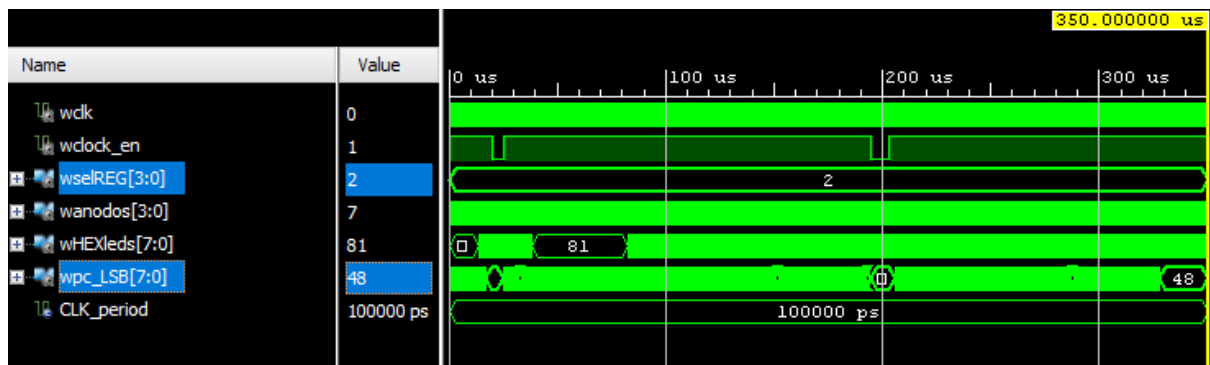
Realizaremos el análisis del circuito una vez le hayamos asignado "Set as Top".



5.2- Resultados de la simulación del circuito "MIPS_Interface" (una captura de pantalla)

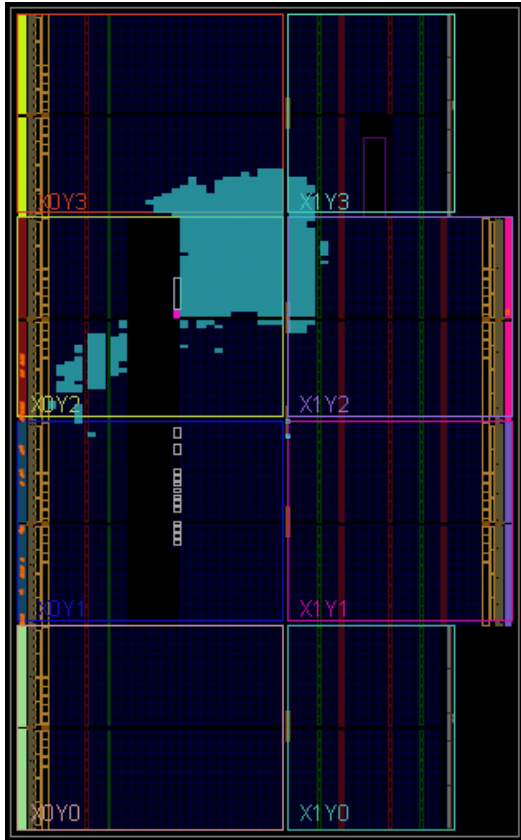
Para poder analizar el circuito implementaremos el fichero de testbench "TB_Mips_interface.vhd" (que contiene los dos contadores "cont_div_freq_refresh" y "cont_pausa_gen") tendremos que cambiar el valor genérico en las sentencias de instalación de los componentes "cont_div_freq_refresh" y "cont_pausa_gen".

Cambiando así el tiempo de simulación a 350us para poder observar la ejecución del sistema



5.3- Captura del resultado de la síntesis e implementación (“Implemented Design”), mostrando la utilización de recursos en la FPGA

Archivo “MIPS_interface.bit” con el que se configura la FPGA, generado tras procesar la síntesis e implementación del archivo "mips_interface.bit".

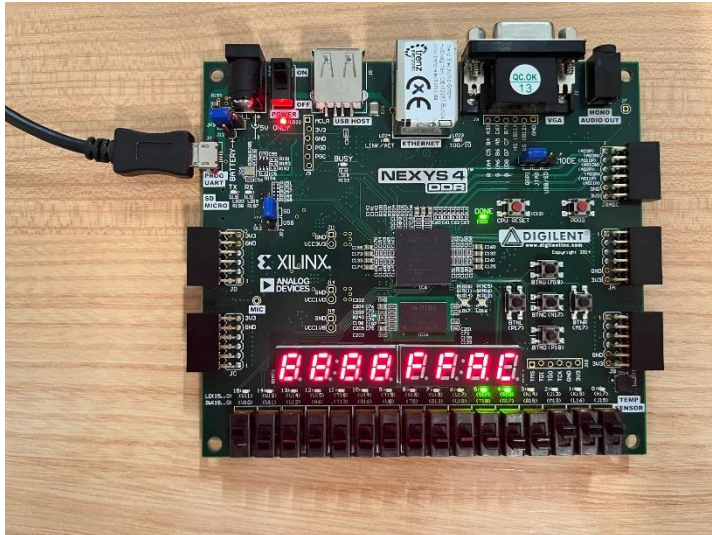


5.4- FPGA en las que aparezca visualizados:

1) el contenido del registro que almacena la suma de los elementos del array1

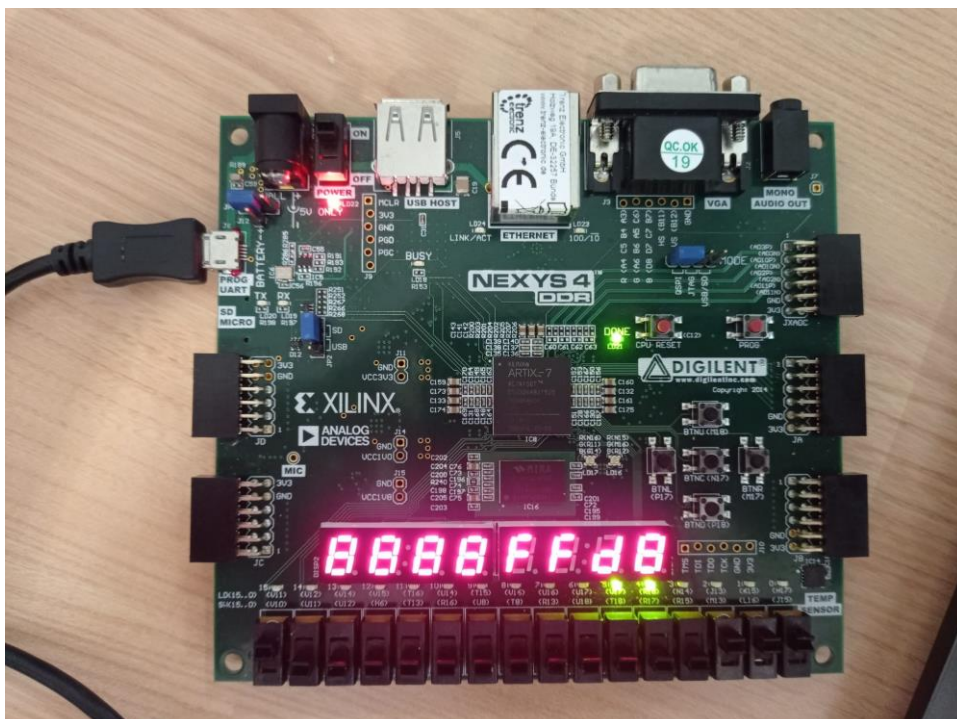
Contenido del registro \$7 del alumno Jorge Méndez:

\$7 = -84 => (FFAC)hex



Contenido del registro \$7 del alumno José Manuel Pecero:

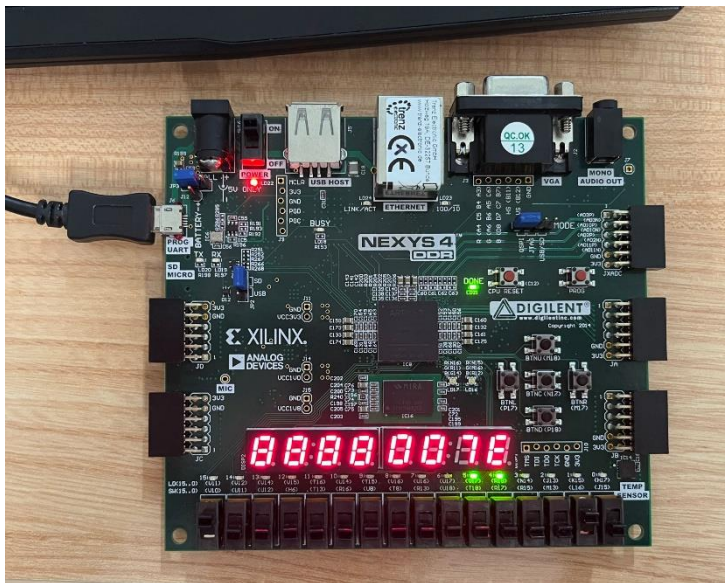
\$7 = -40 = FFD8 hex



2) el contenido del registro que almacena la OR de los valores absolutos de los elementos del array1.

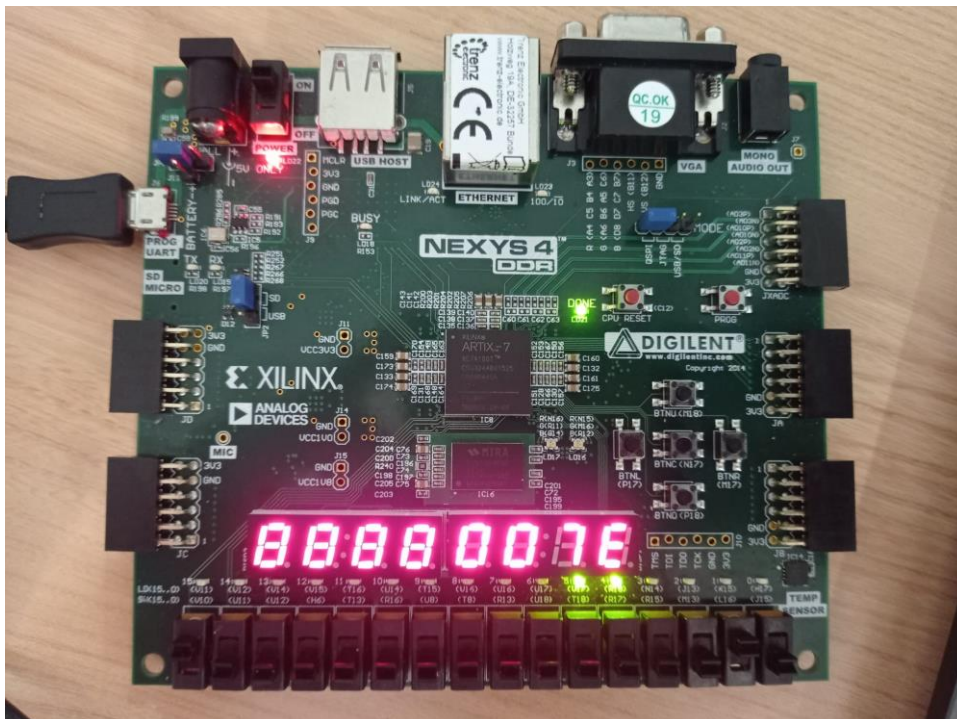
Contenido del registro \$2 del alumno Jorge Méndez:

\$2 = 126 => (007E)hex



Contenido del registro \$2 del alumno José Manuel Pecero:

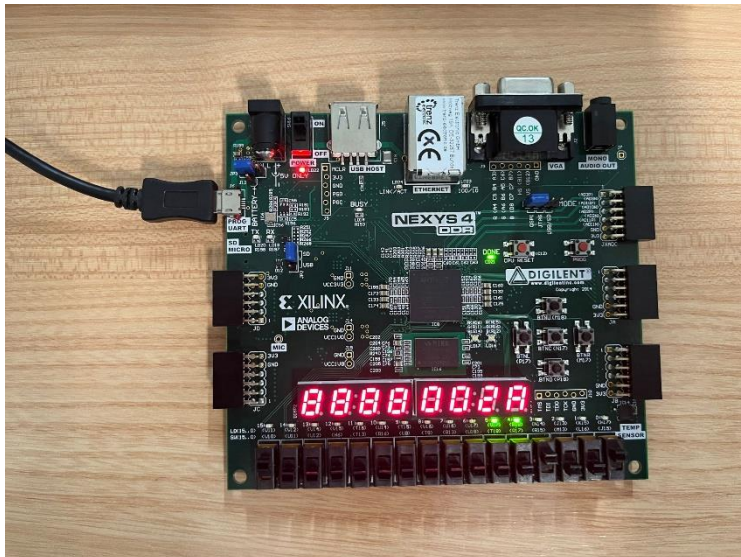
\$2 = 126 = 007E hex



3) el contenido del registro que almacena la suma de 1) y 2).

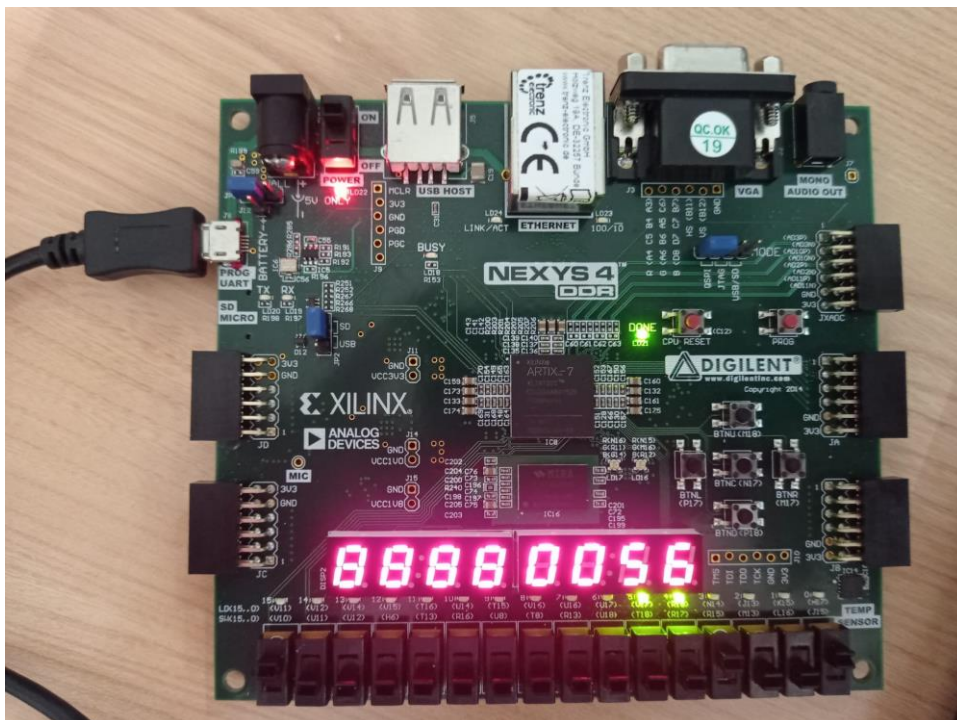
Contenido del registro \$9 del alumno Jorge Méndez:

\$9 = 42 => (002A)hex



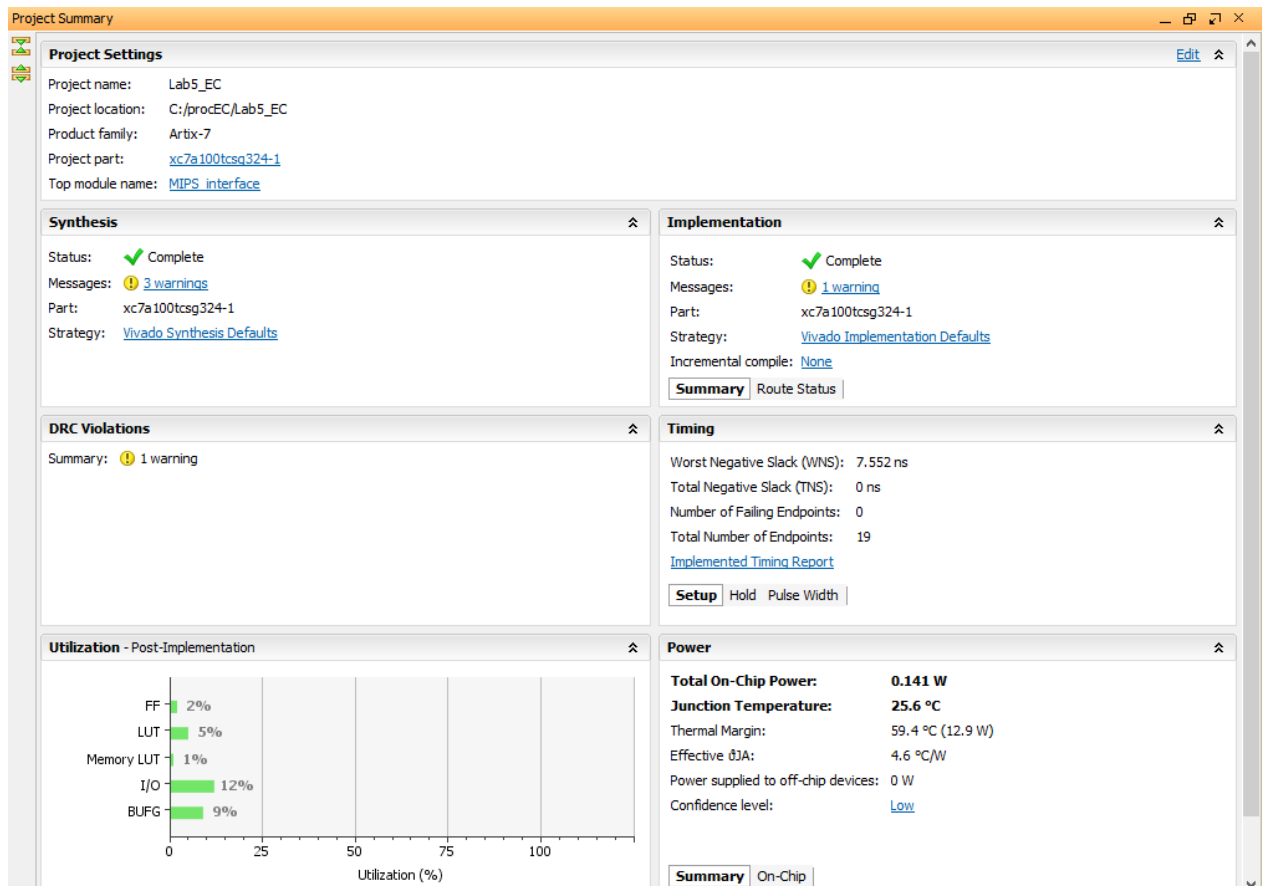
Contenido del registro \$9 del alumno José Manuel Pecero:

\$9 = 86 = 0056 hex



5.5- Resumen final del proyecto

Por último, obtenemos el resumen de todo el proyecto con importantes datos sobre las síntesis, implementaciones y tiempos de ejecución.



5.6- Pregunta de la sesión:

Aplicaciones de las FPGA's en inteligencia artificial (matching learning y deep learning).

Las FPGA son dispositivos programables que permiten la implementación de algoritmos de inteligencia artificial de manera altamente personalizada, algunas de las aplicaciones de las FPGA's en inteligencia artificial son:

- **Bajo consumo de energía:** Las FPGA pueden ser más eficientes en energía que en las GPU, especialmente en implementaciones específicas y altamente optimizadas para tareas de inteligencia artificial.
- **Flexibilidad:** Las FPGA's pueden ser reconfiguradas fácilmente para conseguir una mayor flexibilidad en el desarrollo y la experimentación.
- **Tiempo de respuesta rápido:** Debido a su arquitectura paralela y la capacidad de implementar algoritmos específicos en hardware, las FPGA pueden ofrecer tiempos de respuesta muy rápidos.
- **Implementaciones embebidas:** Las FPGA pueden integrarse fácilmente en sistemas embebidos, lo que las hace ideales para aplicaciones de inteligencia artificial en dispositivos, vehículos autónomos...

Concretamente en el contexto de matching learning y deep learning las FPGA's pueden usarse para acelerar distintas tareas como: Inferencia (Implementación de modelos entrenados en hardware para realizar predicciones en tiempo real), Entrenamiento o Preprocesamiento de datos.

Webgrafía utilizada para la pregunta:

Bajo consumo de energía:

- <https://arxiv.org/pdf/1906.11879>
- <https://www.digikey.es/es/articles/use-the-fpga-fast-path-to-building-edge-ai-applications>

Flexibilidad:

- <https://www.digikey.es/es/articles/use-the-fpga-fast-path-to-building-edge-ai-applications>

Tiempo de respuesta rápido

- <https://www.intel.la/content/www/xl/es/software/programmable/fpga-ai-suite/overview.html>

Implementaciones embebidas:

- <https://www.digikey.es/es/articles/use-the-fpga-fast-path-to-building-edge-ai-applications>
- <https://www.intel.la/content/www/xl/es/automotive/products/programmable/overview.html>

Tareas de Machine Learning y Deep Learning:

- <https://riunet.upv.es/bitstream/handle/10251/179314/Ferrando%20-%20Desarrollo%20de%20Soporte%20de%20Procesos%20de%20Inferencia%20en%20FPGAs%20con%20High%20Level%20Synthesis.pdf?sequence=1>

- <https://www.intel.la/content/www/xl/es/software/programmable/fpga-ai-suite/overview.html>
- <https://www.odbms.org/wp-content/uploads/2014/03/Data-Processing-on-FPGAs.pdf>

Conclusiones

En conclusión, sacamos de esta práctica el poder aplicar los conceptos vistos en clase para realizar un proyecto real, modelando, simulando e implementando un procesador MIPS en FPGA. Todo ello gracias al desarrollo de algunas habilidades como la de programación, aprendiendo así un nuevo lenguaje, el VHDL.

A todo ello hay que sumar las preguntas opcionales de cada sesión que desafían al estudiante y ayudan a que se entiendan realmente esos conceptos vistos durante el laboratorio, en general muy útiles.

Por último, pensamos que realmente este tipo de proyectos son esenciales para comprender la asignatura más allá del papel, combinando así la teoría y la práctica para un mejor aprendizaje.